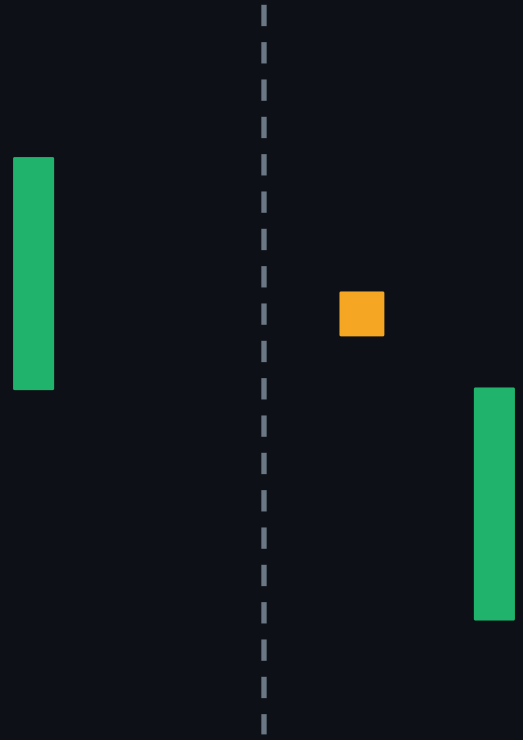
Pong em Assembly

Um jogo x86 de 16 bits para mostrar como CPU, memória, teclado e vídeo trabalham juntos



Arquitetura de Computadores · UFMA

CamScanner

O que quisemos mostrar

Construímos o clássico Pong em Assembly para entender, na prática, como um programa conversa com o hardware.

Até um jogo simples depende de vários componentes trabalhando em conjunto.

1. **Registradores**
2. **Memória e endereçamento**
3. **Dados em baixo nível**
4. **Interrupções**
5. **Entrada e saída**
6. **Memória de vídeo**

CamScanner

Ferramentas que usamos

NASM

MONTADOR

Converte o Assembly em instruções de máquina e gera o executável .COM.

DOSBox

EMULADOR

Cria um ambiente DOS emulado para rodar o jogo.

VGA

VÍDEO

Exibe os gráficos em 320 × 200 pixels, com 256 cores.

Intel 8086

REFERÊNCIA

CPU histórica de 16 bits usada para entender a arquitetura x86.

Usamos instruções x86 de 16 bits; isso não significa que o DOSBox emule apenas um 8086.

CamScanner

Como o jogo funciona

1

Inicialização

Configura vídeo, memória e teclado.

2

Entrada

Identifica as teclas pressionadas.

3

Lógica

Calcula o movimento da bola e das raquetes.

4

Renderização

Desenha tudo no Back Buffer.

5

Exibição

Copia a imagem para a memória VGA.

O ciclo se repete enquanto o jogo estiver em execução.

CamScanner

O processador em ação

```
mov ax, [ball_dx]
add [ball_x], ax
```

mov: copia a velocidade horizontal da bola para o registrador AX.

add: soma esse valor à posição horizontal da bola.

EXEMPLO

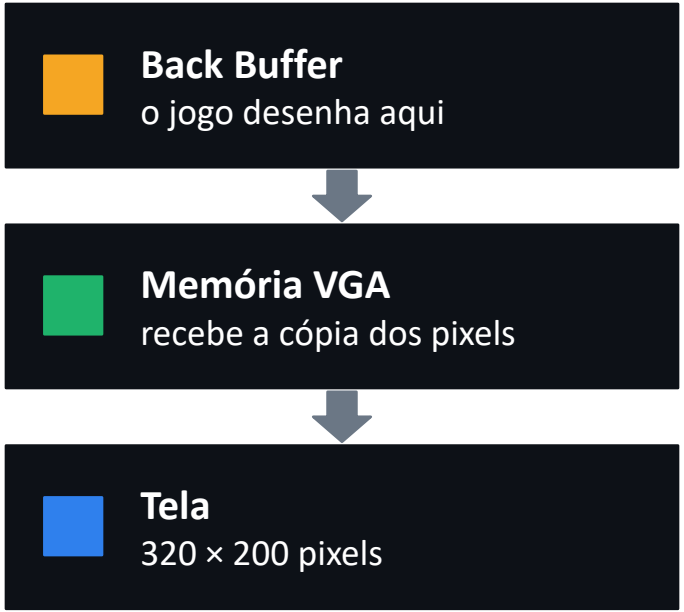
100 + 2

= 102

Posição 100, velocidade 2: no próximo quadro a bola está na posição 102.

CamScanner

Onde cada dado fica



Registradores	Valores temporários na CPU
Memória do programa	Instruções e variáveis
Back Buffer	Prepara a imagem do jogo
Memória VGA	Pixels exibidos na tela
Pilha (Stack)	Dados e registradores temporários

Em vez de desenhar direto na tela, preparamos a imagem na memória e só depois a transferimos para a VGA.

CamScanner

Como o teclado controla o jogo

Jogador 1

W S — sobe / desce

Jogador 2

↑ ↓ — sobe / desce

1. **Tecla pressionada**
   gera um evento de hardware
2. **INT 9**
   a rotina de teclado é chamada
3. **Porta 60h**
   lê o scan code da tecla
4. **Lógica do jogo**
   move a raquete correspondente

CamScanner

Colisões e pontuação

A bola pode colidir com:

1. Paredes superior e inferior
2. Raquetes dos jogadores
3. Laterais da tela

Quando a bola passa de uma raquete, o adversário marca ponto.

5

pontos para vencer a partida

CamScanner

O que o projeto demonstra

CPU	Executa as instruções Assembly	**Registradores**	Manipulam posições e velocidades
Memória RAM	Guarda variáveis e Back Buffer	**Interrupções**	Capturam eventos do teclado
Portas de E/S	Falam com teclado e vídeo	**Segmentação**	Organiza o acesso à memória
Assembly	Controla as operações do programa	**VGA**	Apresenta os gráficos do jogo

CamScanner

Um jogo simples, vários componentes trabalhando juntos

1

Fluxo de execução

o ciclo que mantém o jogo vivo

2

Registradores e memória

onde posições e imagens vivem

3

Interrupções

e Back Buffer: entrada rápida e vídeo sem falhas

Obrigado! Perguntas?

CamScanner